

Execute M5rules against CarenR

⚙ Status	In progress
🕒 Last edited time	March 19, 2026 11:00 PM

▼ Execution

The script is at the same location as `ripper_pipeline.R`. A few things worth noting about how it differs:

Key differences from RIPPER pipeline:

- `Wine_mhl` stays **continuous** — no tertile cuts, no class labels
- Evaluation uses **MAE, RMSE and R^2** instead of accuracy/Kappa
- A **naive baseline** is computed first (always predict the training mean) — any model must beat this to be useful
- The script prints how much M5Rules **improves over baseline** in percentage terms, which is a clean metric for your thesis

What M5Rules outputs:

Each rule will look like:

```
IF condition1 AND condition2 THEN
  Wine_mhl = a*feature1 + b*feature2 + intercept
```

So instead of "predict Low" like RIPPER, it gives an actual numeric prediction from a small linear model. This makes it directly comparable to CarenR's subgroup means.

Run it with `file <- 1` for RDD first, then `file <- 2` for RVV. Share the output and we'll do the same feature comparison we did for RIPPER.

▼ M5rules results

Looking at the M5Rules RDD output, here's the comparison with CarenR:

M5Rules RDD — Feature Overlap with CarenR RDD

CarenR found 5 rules, all anchored to the **Flowering stage** with these features:

```
swa_fl_y1_a15 , tm_fl_y1_a10 , tm.days.less.15_fl_y1_b15 ,  
days.rf.above.1mm_fl_y1_c15 , tm_fl_y1_c15
```

M5Rules found 1 linear rule with 8 features:

```
tm_fl_y1_c15 , tm_fl_y0_b10 , tm_bb_y0_a30 , tm_hv_y1_c10 , tn_hv_y0_a30 ,  
tm.days.less.15_fl_y1_b15 , iaf_fl_y0_b10 , iaf_fl_y0_b20
```

Shared features (2/5 from CarenR):

- `tm_fl_y1_c15` — mean temperature at flowering, current year (late window)
- `tm.days.less.15_fl_y1_b15` — cold days count at flowering, current year (mid window)

Both methods agree on the same climate theme: **flowering-stage temperature in the current year is the dominant signal for RDD**. M5Rules assigns `tm_fl_y1_c15` a positive coefficient (+35.10) — warmer flowering → more production — and `tm.days.less.15_fl_y1_b15` a negative coefficient (−16.20) — more cold days → less production. This is directionally identical to CarenR's RDD rules (cool, wet flowering = Low production).

What M5Rules adds that CarenR didn't use:

- `iaf_fl_y0_b10` (+2811.44) and `iaf_fl_y0_b20` (−2723.84): LAI at flowering in the *previous year*, two adjacent windows. The coefficients nearly cancel, making this essentially a vine-leaf-index *differential* — a previous-year vine stress signal.
- `tm_fl_y0_b10` , `tm_bb_y0_a30` , `tm_hv_y1_c10` : temperature carry-over from previous year and harvest-stage temperature.
- `tn_hv_y0_a30` : night temperature at harvest previous year — same feature RIPPER also selected for RDD.

What CarenR used that M5Rules ignored:

- `swa_fl_y1_a15` — soil water availability at flowering (moisture/stress signal)
- `tm_fl_y1_a10` — early-window flowering temperature
- `days.rf.above.1mm_fl_y1_c15` — rainy days at flowering (precipitation-based signal)

Comparison summary across all three methods for RDD:

Feature	CarenR	RIPPER	M5Rules
tm_fl_y1_c15	✓	✗	✓
tm.days.less.15_fl_y1_b15	✓	✗	✓
swa_fl_y1_a15	✓	✗	✗
tm_fl_y1_a10	✓	✗	✗
days.rf.above.1mm_fl_y1_c15	✓	✗	✗
tn_hv_y0_a30	✗	✓	✓
iaf_fl_y0_b10/b20	✗	✗	✓

Bottom line on RDD: M5Rules aligns better with CarenR than RIPPER did (2 shared features vs 0), converging on the flowering temperature signal. However, its CV $R^2 = -0.076$ — the model memorises patterns (training $R^2 = 0.403$) but fails to generalise to unseen years. RIPPER had the same generalisation problem for RDD. This reinforces that RDD is a harder region to model, with CarenR's distribution-based rules appearing more stable precisely because they don't try to predict a number — they describe subgroup behaviour.

Next up: run M5Rules with `file <- 2` for RVV. That's where the interesting comparison will be — RVV has the LAI signal dominant in CarenR (iaf_hv_y1_c10), and M5Rules may either pick that up or go a different direction entirely. Want me to switch the config and share the RVV output?

Great results — now let's break down what M5Rules found for RVV and how it compares with CarenR.

M5Rules RVV — Structure

This time M5Rules discovered **2 rules** instead of 1 (unlike RDD), which is already more interesting structurally:

Rule 1 (32 obs, ~40% of years): fires when `rf_hv_y1_a10 ≤ 38.63` AND `tm_sm_y0_a20 > 20` — low harvest rainfall *and* warm previous-year summer. Leads to a 12-variable linear model involving temperatures at flowering/maturation/harvest, extreme-heat days, and multiple soil water features.

Rule 2 (48 obs, ~60% of years — the default/else rule): everything that doesn't meet Rule 1 conditions. Surprisingly simple — just two features: `tm_fl_y0_b10`

(prev-year flowering temperature) and `swa_fl_y1_a15` (current-year soil water at flowering).

Feature Overlap: M5Rules RVV vs CarenR RVV

CarenR RVV is dominated by LAI at harvest (`iaf_hv_y1_c10`) across 22 rules, with supporting features `tm_sm_y0_a20` , `tn_hv_y0_a30` , `swa_hv_y1_a20` .

Feature	CarenR RVV	RIPPER RVV	M5Rules RVV
iaf_hv_y1_c10 (LAI harvest — dominant)	✓	✗	✗
tm_sm_y0_a20	✓	✓	✓ (condition + model)
tn_hv_y0_a30	✓	✓	✗
swa_hv_y1_a20	✓	✓	✓ (Rule 1 model)
rf_hv_y1_a10	✗	✗	✓ (Rule 1 condition)
tm_hv_y1_c10	✗	✗	✓ (Rule 1 model)
swa_fl_y1_a15	✗ (CarenR RDD)	✗	✓ (both rules)

Shared with CarenR: 2 features — `tm_sm_y0_a20` and `swa_hv_y1_a20` . Same count as RDD (2/5), but here they're secondary features in CarenR — **the dominant signal, LAI, is completely absent from M5Rules again.**

The fact that neither RIPPER nor M5Rules ever selects `iaf_hv_y1_c10` is striking. CarenR identifies LAI at harvest as the single most powerful subgroup discriminator for RVV, but both regression/classification learners ignore it entirely. This isn't a coincidence — it likely reflects a fundamental difference in what these methods optimise for. CarenR finds where the *distribution shape* of Wine_mhl shifts; M5Rules minimises prediction error across all years and LAI probably has too much noise in the tails to reduce MAE consistently.

Generalisation: RVV is even harder than RDD

Metric	RDD	RVV
Training R ²	0.403	0.529
CV R ²	-0.076	-0.162
CV MAE vs baseline	+0.7%	-7.0%
CV RMSE vs baseline	-3.5%	-10.5%

RVV's CV performance is *worse than always predicting the mean* — M5Rules actually hurts prediction here. Training $R^2=0.529$ looks decent but the collapse to CV $R^2=-0.162$ shows severe overfitting with 80 observations and 59 features. Rule 1's 12-variable linear model is effectively memorising 32 data points.

Cross-method summary for RVV

The convergence story here is weaker than for RDD:

- RIPPER and CarenR agree on 3 features (tm_sm_y0_a20, tn_hv_y0_a30, swa_hv_y1_a20)
- M5Rules agrees with CarenR on 2 features (tm_sm_y0_a20, swa_hv_y1_a20)
- All three agree on **tm_sm_y0_a20** (previous-year summer temperature) — this is the one feature all methods consistently identify for RVV
- LAI (iaf_hv_y1_c10) is CarenR-exclusive and appears in none of the other methods

Previous-year summer temperature (tm_sm_y0_a20 > 20 as M5Rules Rule 1 condition, and present in CarenR and RIPPER) is the most robust cross-method signal for RVV. It makes agronomic sense: a warm summer in year Y-1 builds vine reserves that carry into year Y production.

Where does this leave the prediction pipeline?

Both RIPPER and M5Rules fail to generalise on these datasets — small N, high feature count, temporal structure that k-fold CV doesn't fully respect. For the walk-forward validation milestone this actually strengthens your argument: rule-based generalisation requires methods that are designed for distribution discovery rather than error minimisation. CarenR's rules (especially the LAI signal in RVV) represent domain-stable patterns that survive the temporal split better precisely because they describe subgroup membership rather than trying to fit a regression surface.

Ready to move on to the walk-forward validation design? The comparison across CarenR, M5Rules, Decision Tree and naive baseline will be the empirical test of that argument.